DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Java – Beautifully Broken

JDK 28 recycles your tiny native allocations, and Helidon starts marching to the JDK's six-month beat

Issue #13 | Sat, 17 Oct 2026 | ⏱️ ~38 min read | Level 3/5

☕ **Today's two stories**

1. **JDK 28 makes the most common Foreign Function & Memory pattern up to ~19x cheaper, without any change to your code.** Small allocations from `Arena.ofConfined()` now come from a reusable 64-byte native pool held by each thread, instead of a fresh `malloc` and `free` every time. On 14 October the JDK deleted its own internal workaround for the old cost. We read the code, explain how the pool works and where its limits are, and ran the old pattern on JDK 21 to see what the overhead looks like.
2. **Helidon 27 is Helidon's first release under the OpenJDK "Tip & Tail" release model, and the shake-up is bigger than a version number.** It requires Java 27, no longer includes MicroProfile in the main release, moves its integrations into a separate repository and adds new Messaging and Data modules. Helidon 4.5 stays on as the long-term "tail". Here's what Tip & Tail (JEP 14) means, what changed, and how to decide whether to stay or move.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 JDK 28 recycles your tiny native allocations

JDK 28 · CORE LIBRARIES · FFM API · JDK-8385697 · INTEGRATED 3 SEP 2026, EXPLAINED BY ORACLE 5 OCT · ~14 MIN READ

You call a C function and it needs somewhere to write four bytes: an error code, a file descriptor, a single int. In Java's modern native-interop API, the safe way to do that is to open a tiny memory "arena", allocate four bytes, make the call and close the arena. It's clean and it can't leak. Until now it also meant a trip to the C allocator and back for every call, and that bookkeeping could cost far more than the four bytes it was for. JDK 28 makes that pattern almost free, and the JDK's own developers have already deleted the workaround they built because of it.

WHAT HAPPENED

On 3 September 2026, Oracle engineer Per-Åke Minborg integrated **JDK-8385697, "Add a Pooled Confined Arena"** into the JDK 28 main line, reviewed by Jorn Vernee and Alan Bateman.[1] It touches 17 files (1,960 lines added, 18 removed), most of them tests, and adds one new internal class, `jdk.internal.foreign.ConfinedSegmentPool`.[1] We checked the build tags: the change isn't in JDK 28 early-access build 14 (tagged 3 September) but it is in build 15 (10 September) and every build since.[2]

On 5 October, Minborg explained the change on Inside.java, Oracle's Java team blog.[3] The summary: starting with JDK 28, **small allocations made through `Arena.ofConfined()` can be served from reusable native-**

memory pools kept by each thread, and "no source code changes" are needed.[3] His development benchmarks show speedups of 6.8x to 18.9x for 5- and 20-byte allocations, depending on the platform.[3]

Then, on 14 October, a follow-up commit, **JDK-8392541**, removed two internal JDK classes, `BufferStack` and `CaptureStateUtil`, deleting 1,526 lines.[4] `BufferStack` was a thread-local recycling stack the JDK's own native-call machinery used to avoid exactly the cost that the pool now removes. Its replacement is a plain `Arena.ofConfined()`.[4] When the platform team throws away its own optimisation because the public API is now fast enough, that's a fairly strong signal.

JDK 28 is due in March 2027. Nothing in the public API changed: we compared `Arena.java` and the pooling commit, and the change lives entirely in internal classes.[1]

THE BACKGROUND YOU NEED

Calling C from Java, then and now

Java programs sometimes need to call code written in C: an operating-system function, a compression or crypto library, a GPU driver. For 25 years the official way was **JNI** *≈ the Java Native Interface (1997), the original way to call C/C++ code from Java. It needs a Java `native` method, a generated C header and hand-written C glue code*. JNI works, but it's painful. JEP 442 lists the problems: you maintain three artifacts across two toolchains (the Java declaration, a generated header, a C implementation), there's no mapping between Java objects and C structs, and developers often fall back on `sun.misc.Unsafe` to pass raw addresses around.[5] Off-heap memory had its own trouble: a direct `ByteBuffer` tops out at about 2 GB, has a rudimentary API and is freed only when the garbage collector gets around to it.[5]

The replacement is the **FFM API** *≈ the Foreign Function & Memory API (package `java.lang.foreign`): call native functions and work with native memory from pure Java, with no C glue code. Final since JDK 22*, built in **Project Panama** *≈ the OpenJDK project that improves the connection between Java and native code*. It took a long road. A foreign-memory API first incubated in Java 14 (JEP 370) and a foreign linker in Java 16 (JEP 389). JEP 412 merged them in JDK 17.[6] A second incubator came in JDK 18 (JEP 419),[7] then previews in JDK 19 (JEP 424), JDK 20 (JEP 434) and JDK 21 (JEP 442).[5] JDK 22 finalised it in March 2024 with JEP 454, "after two rounds of incubation and three rounds of preview", in InfoQ's words.[8]

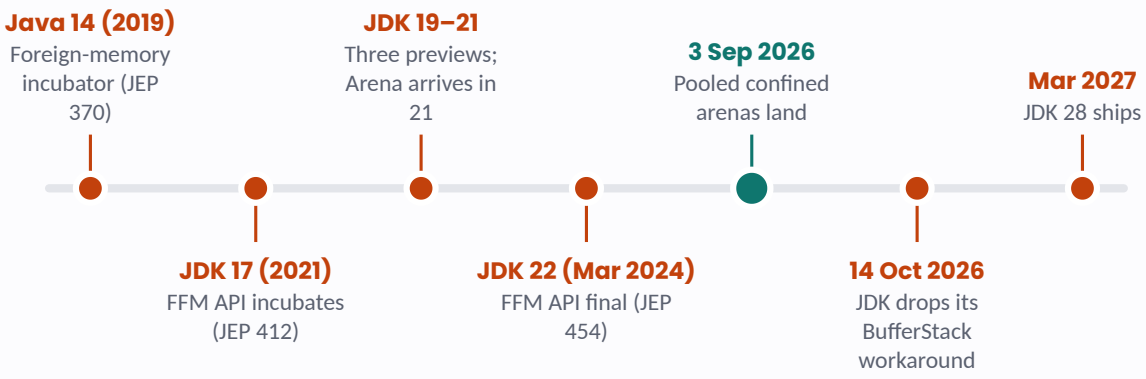
Arenas: who owns this memory, and for how long?

The FFM API's central idea is that every piece of native memory, a **MemorySegment** *≈ a Java object that describes a contiguous region of memory (native or on the heap) with a size and a lifetime, and checks every access against both*, belongs to an **Arena** *≈ the FFM object that controls when native memory is freed. Every segment you allocate belongs to an arena, and closing the arena frees it all at once*. The JDK offers four kinds, and JEP 442 describes them like this:[5]

- **Global**: the memory lives forever and is never freed.
- **Automatic** (`Arena.ofAuto()`): freed by the garbage collector some time after the segment becomes unreachable, so you don't control when.

- **Confined** (`Arena.ofConfined()`): freed deterministically when you close it, and only the thread that opened it may touch its segments. Use after close throws an exception.
- **Shared** (`Arena.ofShared()`): any thread may access and close it, but closing needs extra synchronisation, so the memory may not be freed immediately.

The long road to fast native interop



Sources: JEP 412, JEP 442, foojay (JEP 419), InfoQ (19 Mar 2024), openjdk/jdk commits e9222ab and 7f0d1d9.

A **confined arena** ≈ *an arena that only its creating thread may use, freed the moment you close it. Usually used with try-with-resources* is the one you reach for around a single native call, because it's the cheapest to close and it pairs perfectly with try-with-resources. Here's a real example: calling the POSIX `open` function and reading `errno`, the C error code, if it fails. This needs a little scratch memory for the path string and for the captured error code.

```
// JDK 22+ (final API). On JDK 21: --enable-preview, allocateUtf8String
Linker linker = Linker.nativeLinker();
StructLayout STATE = Linker.Option.captureStateLayout();
long ERRNO = STATE.byteOffset(PathElement.groupElement("errno"));
MethodHandle open = linker.downcallHandle(
    linker.defaultLookup().find("open").orElseThrow(),
    FunctionDescriptor.of(JAVA_INT, ADDRESS, JAVA_INT),
    Linker.Option.captureCallState("errno"));

try (Arena arena = Arena.ofConfined()) {
    MemorySegment state = arena.allocate(STATE);           // 4 bytes
    MemorySegment path  = arena.allocateFrom("/no/such/file"); // 14 bytes
    int fd = (int) open.invokeExact(state, path, 0);
    if (fd < 0) System.out.println("errno = " + state.get(JAVA_INT, ERRNO));
}
```

We compiled and ran the JDK 21 version of this (identical except for `allocateUtf8String` and the preview flag) on OpenJDK 21.0.12.1. It prints `errno = 2`, which is `ENOENT`, "no such file". The arena allocated 18 bytes in total.[10] Minborg's post says this pattern is common precisely for handling `errno` in system calls.[3] It's also everywhere in real libraries. **JLine**, a widely used Java library for console input, has an FFM terminal backend

that opens 27 confined arenas across its main source, typically to hold one small C struct such as the 8-byte window-size record for an `ioctl` call, or a `pollfd` plus a captured `errno`.[11]

Why tiny allocations were expensive

Before JDK 28, every `arena.allocate(...)` on a confined arena took the general path. We read it in the JDK source: it calls `Unsafe.allocateMemory` (the JDK's wrapper around the C allocator, `malloc`), zeroes the memory, and then creates a small cleanup object and adds it to the arena's resource list. When you close the arena, the JDK walks that list and calls `freeMemory` for each allocation.[9] So a 4-byte scratch value costs a `malloc`, a `free`, one extra Java object and some list manipulation. As Minborg puts it, for small values "this bookkeeping could cost more than accessing the memory".[3]

We measured the shape of that overhead on JDK 21 in our sandbox (two vCPUs, a simple warmed-up loop, not a JMH benchmark, so treat it as indicative). Opening a confined arena, allocating one `int`, writing and reading it and closing the arena took about **134–141 ns per iteration**. Opening and closing an arena that allocates nothing took about **0.4 ns**, and writing and reading a reused segment took about **0.6 ns**.[10] The cost isn't the arena or the memory access. It's the allocation round-trip. (Our numbers come from a different JDK and different hardware from Oracle's, so don't compare them directly. Oracle's non-pooled baseline on an Apple M4 is about 16 ns.[3])

How small are typical allocations? Minborg instrumented a test-suite run. Some confined arenas allocated no native memory at all, and **more than 99.99% of those that did allocate used less than 64 bytes**. He stresses that a test suite isn't representative of every application, but the distribution favoured "a small, lazy pool".[3] That's exactly what was built.

HOW IT WORKS

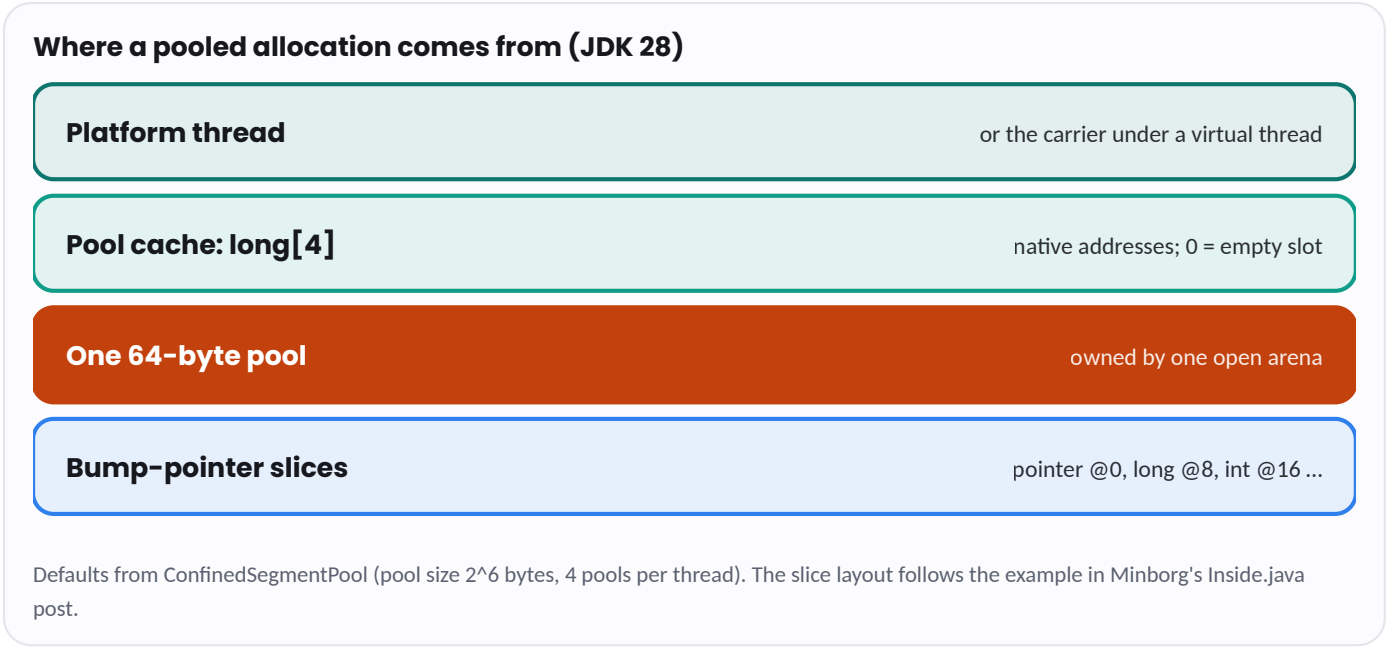
A small per-thread cache of 64-byte pools

The new class's own documentation describes the design. Each platform thread lazily keeps a cache with a configurable number of fixed-size pools. A confined arena carves small allocations out of one pool, and anything that doesn't fit uses the regular allocator.[1] The defaults are set in the code: pools are **2^6 = 64 bytes** and the cache holds **4 pools per thread**.[1] The cache itself is just a `long[]` of native addresses stored in a new field of `java.lang.Thread`, where 0 means an empty slot.[1]

Here's the life of one arena, from the source of `ArenaImpl.OfConfined` and `ConfinedSegmentPool`:[1]

1. **Opening is free.** `Arena.ofConfined()` doesn't touch the pool. An arena that never allocates never acquires one, so the many arenas that allocate nothing pay nothing.[3]
2. **The first small allocation acquires a pool.** If both the size and the alignment are at most 64 bytes, the arena takes the first available pool out of the thread's cache and removes it from the cache, so it now belongs exclusively to this arena. If the cache is empty (for example, the very first arena on a thread, or a fifth nested arena), the arena `malloc`s its own 64-byte "local" pool and zeroes it.

3. **Allocations are bump-pointer slices.** The arena keeps an offset into its pool. Each allocation aligns the offset up, checks the slice still fits in 64 bytes, and moves the offset forward. That's a few arithmetic instructions: no `malloc`, no cleanup object. If the slice doesn't fit, or the request is bigger than 64 bytes, it silently falls back to the regular path.
4. **Closing clears and returns the pool.** The arena first runs any ordinary cleanup actions, then zeroes only the part of the pool it used and puts the pool back in a free slot of the thread's cache. If the cache is full, the pool is freed instead.



The zeroing is how the JDK keeps a promise that FFM always made: newly allocated native segments contain zeroes, so one arena can never see bytes left behind by another.[3] New pools are zeroed when created, and on release only the used prefix needs clearing. For pools of 64 bytes or less, the clearing code is an unrolled `switch` that writes up to eight 8-byte zeros without a loop.[1]



There's one more subtle rule in step 4. A cleanup action registered by your code might still read the pooled memory, so the pool is cleared and released only *after* all regular cleanup actions have run. The code comment says exactly this: "clear and release the pool only after they have run".[1]

Virtual threads borrow from their carrier

A per-thread cache is a problem for **virtual threads** ≈ *lightweight threads (final in JDK 21) managed by the JVM rather than the OS. Millions can exist at once, and each runs on top of an ordinary "carrier" platform thread*. Giving every one of potentially millions of virtual threads its own cache would undo their small footprint.[3] So a virtual thread takes its pool from the cache of the **carrier thread** ≈ *the platform thread a virtual thread is currently running ("mounted") on. A virtual thread can move to a different carrier whenever it blocks* it's running on. In code, the arena briefly **pins** ≈ *stops a virtual thread from being unmounted from its carrier, so it stays on the same platform thread for a moment* the virtual thread with `Continuation.pin()`, takes the pool from the carrier's cache, and unpins it.[1]

Because the pool is removed from the cache when acquired, it now belongs to the arena, not the carrier. The virtual thread can migrate to another carrier while the arena is open, and if the original carrier thread dies meanwhile, it can't free a pool that's still in use. On close, the pool goes back into the cache of whichever carrier the virtual thread is on *at that moment*.[3] On Oracle's M4 numbers, a pooled 5-byte allocation from a virtual thread takes 3.2 ns against 1.05 ns from a platform thread, still about five times faster than the 15.7 ns non-pooled path.[3]

The cache needs no locks. Only its owning platform thread ever touches it, either directly or while acting as a carrier, so acquire and release are plain array operations.[1] When a platform thread terminates, the JDK frees whatever pools are sitting in its cache. Minborg's commit also reorders thread shutdown so this happens *after* terminating thread-locals run, because those callbacks might themselves use confined arenas and return pools.[1]

The numbers, and the 64-byte cliff

Minborg's benchmark tables show where the win is and where it stops.[3]

Confined-arena allocation on an Apple M4, platform threads (ns/op, lower is better)



Source: Per-Åke Minborg, Inside.java, 5 Oct 2026 (JMH average time, Apple M4, macOS). These are Oracle's development benchmarks, not independent measurements. 100 bytes exceeds the 64-byte pool and takes the regular path.

The headline speedups for 5- and 20-byte allocations are 12.3x and 8.6x on Linux AArch64, 6.8x and 7.8x on Linux x64, 18.6x and 16.8x on macOS AArch64, and 16.8x and 18.9x on Windows x64.[3] Above 64 bytes, the

two paths perform the same. That's not a regression, it's the design: big allocations never touch the pool.[3] The post also warns that microbenchmark speedups "do not translate directly" to whole applications. The gains matter most when allocating, creating the arena and cleaning up are a big share of a short native call.[3]

Why the JDK deleted BufferStack

Before this change, the JDK's own native linker had worked around the cost internally. When a native call needed a temporary buffer (for example for some struct arguments or return values), the linker pushed a "frame" on `BufferStack`, a thread-local 256-byte recycling stack, sized by an internal property, `jdk.internal.foreign.LINKER_STACK_SIZE`.[4] The 14 October commit deletes `BufferStack` and its tests and benchmarks. The linker's `newBoundedArena(size)` now simply opens an `Arena.ofConfined()`, allocates the requested size from it, and slices from that.[4] The public, general-purpose mechanism replaced a private, special-purpose one. That's good news for everyone else: the JDK isn't keeping a faster path to itself.

WHAT IT MEANS FOR YOU

If you don't call native code, nothing changes. Most Spring and Jakarta EE developers never touch `java.lang.foreign` directly. You may benefit indirectly as libraries add FFM backends. JLine's `terminal-ffm` module is one example.[11]

If you write FFM code, keep writing it the obvious way. The safest pattern, a short-lived confined arena per call, is now also the fast one for small scratch values. You don't need hand-rolled thread-local buffer caches or long-lived arenas reused across calls to dodge allocation costs, and the JDK itself has just stopped doing that.[4] Code generated by **jextract** *≈ the OpenJDK tool that reads a C header file and generates the Java FFM bindings (layouts and method handles) for it* and pointer "out-parameters" are among the cases Minborg says benefit most.[3]

Keep each arena's small allocations under 64 bytes if you can. The pool is a single 64-byte block per arena. Two 40-byte structs don't both fit: the first comes from the pool and the second takes the regular path. One big allocation followed by small ones is fine, because the big one bypasses the pool and doesn't use it up.[1] Alignment matters too: a request aligned to more than 64 bytes always takes the regular path.

Shared and automatic arenas are unchanged. Only confined arenas pool.[3] If you use `Arena.ofShared()` for per-call scratch space "just in case", switching to `ofConfined()` gets you the pool, as long as only one thread touches the memory.

Memory overhead is tiny, but it's per thread. Each platform thread that uses confined arenas can hold up to four cached 64-byte pools, which is 256 bytes. By our arithmetic, a server with 1,000 platform threads that all do native calls would hold about 250 KB in pools. Virtual threads don't add any, because they borrow from their carriers.[1]

💥 Watch out for

- **`malloc` and `free` no longer match your arenas one to one.** Closing a pooled arena may return memory to the JDK's cache instead of the C allocator. Minborg says applications and diagnostic tools "must not assume" a one-to-one correspondence.[3] If you track native memory with allocator hooks or `LD_PRELOAD` tools, expect fewer, longer-lived blocks.
- **The tuning knobs are internal and unsupported.** The source defines `jdk.internal.foreign.native.confined.pool.power.size` (log2 of the pool size, 3 to 20) and `...power.count` (log2 of the pools per thread, 0 to 3). A negative value disables pooling. The code comment says they come with "no behavioral or compatibility guarantees".[1] Use them for experiments, not production.
- **`-XX:+PageAlignDirectMemory` turns pooling off.** If direct memory must be page-aligned, slicing small segments from a shared pool would break that, so the pool disables itself.[1]
- **This is JDK 28 only.** It's in early-access builds from build 15 onwards and ships in March 2027.[2] There's no sign of a backport. It's an internal optimisation, not a fix.
- **Microbenchmarks aren't applications.** The 6.8x–18.9x figures are Oracle's development benchmarks for a single allocation.[3] Measure your own workload before you attribute a speedup to this.

WHAT HAPPENS NEXT

The code is in the JDK 28 main line, so the next step is simply JDK 28's release in March 2027. Until then you can try it in the early-access builds. Watch the Panama mailing list and JDK bug tracker for tuning of the defaults: the configuration hooks and tests for pool sizes from 8 bytes to 1 MB already exist, which makes it easy to adjust if real-world data argues for it.[1]

📖 Words from this story

JNI: the Java Native Interface (1997), the original way to call C/C++ from Java, using `native` methods and C glue code.

FFM API: the Foreign Function & Memory API (`java.lang.foreign`), for calling native code and managing native memory from pure Java. Final in JDK 22.

Project Panama: the OpenJDK project that improves the connection between Java and native code.

MemorySegment: a Java object describing a region of memory, with a size and lifetime that every access is checked against.

Arena: the FFM object that controls when native memory is freed. Closing it frees every segment it allocated.

Confined arena: an arena only its creating thread may use, freed deterministically on `close()`.

Virtual threads: lightweight JVM-managed threads (final in JDK 21) that run on top of carrier platform threads.

Carrier thread: the platform thread a virtual thread is currently mounted on.

Pin: to keep a virtual thread on its current carrier for a moment.

jextract: the OpenJDK tool that generates Java FFM bindings from a C header file.

STORY 2 Helidon 27 marches to the JDK's six-month beat

FRAMEWORKS · HELIDON · JEP 14 · RELEASED WITH JDK 27 IN SEPTEMBER, INFOQ DEEP COVERAGE 16 OCT 2026 · ~13 MIN READ

Most Java libraries live by one rule: support the oldest JDK your users might still run. That's why so much of the ecosystem only recently left Java 8 behind. Helidon, Oracle's microservices framework, just did the opposite. Version 27 runs only on Java 27, will move up with every JDK release, and leaves its old version line as a slow, stable "tail" for everyone else. It's Helidon's first release under a model that three JDK architects proposed to the whole ecosystem two years ago, and how it goes will tell the rest of us a lot.

WHAT HAPPENED

Helidon 27.0.0 shipped alongside JDK 27 in September. The release was tagged on 17 September, two days after JDK 27's release,[13] and the Helidon team announced it on Inside.java as "the first release under the Tip-and-Tail model used by OpenJDK".[15] On 16 October, InfoQ published the first detailed walkthrough of what changed.[12]

The changelog sets out the new rules in four sentences. Helidon releases "will now align with JDK releases and the minimum Java requirement will bump to match the corresponding JDK release". Helidon 27 is a feature release. **Helidon 4 is considered the LTS release, and Helidon 29 will be the next one.** And "a minimum of Java 27 is required to use Helidon 27".[13]

The structural changes are just as large:[13,14]

- **MicroProfile is out of the main release.** MicroProfile support moved into a separate project, Helidon MicroProfile, which "is not part of the Helidon 27 release" and will release independently. Until it does, the upgrade guide tells MicroProfile users to stay on Helidon 4.5.x.
- **Integrations become "Extensions"**, in their own repository with independent release cycles.
- **HTTP method names are now case-sensitive**: `GET` matches, `get` doesn't.
- **New modules**: Helidon Data JDBC (incubating), Helidon Messaging (preview), and a feature-complete Helidon Declarative (preview).

On the tail side, **Helidon 4.5.5** came out on 15 September. It requires Java 21, recommends Java 25, and is described as "strongly recommended for all users of Helidon 4".[16]

THE BACKGROUND YOU NEED

What is Helidon?

Helidon ≈ *Oracle's open-source Java framework for microservices. It started in 2018 as a lightweight web server plus a MicroProfile implementation* was introduced in September 2018 (its working name was J4C, "Java for Cloud") with two flavours. **Helidon SE** was a small set of libraries (web server, configuration, security) that you wire together yourself. **Helidon MP** implemented **MicroProfile** ≈ *a set of Eclipse Foundation specifications for enterprise Java*

microservices (config, health, metrics, fault tolerance, REST clients), built on Jakarta EE APIs such as CDI and JAX-RS.[12] Helidon 3.0 (2022) moved to JDK 17. In October 2023, Helidon 4.0 replaced its Netty-based server with "Níma", a new web server written for **virtual threads** *≈ lightweight JVM-managed threads, final in JDK 21, that make simple blocking code scale like asynchronous code*.[12]

In March 2026, Oracle put Helidon into its new **Java Verified Portfolio** (JVP), a curated set of Oracle-supported tools and libraries. JVP brought commercial Helidon support to "all Java SE subscribers and OCI customers at no additional cost". Before that, it was limited to WebLogic and Coherence customers.[17]

The problem Tip & Tail solves

Every library maintainer faces the same dilemma. Users who want stability ask for security fixes without new features. Users who want progress ask for new JDK features. Most libraries try to serve both from one release line, and that pleases neither.

In autumn 2024, Alex Buckley, Brian Goetz and Ron Pressler published **JEP 14, "The Tip & Tail Model of Library Development"**.[18] It's an "informational" JEP: a recommendation to the ecosystem, not a feature of the JDK. It names four problems with the usual approach:[18]

- **One size fits all**: every release mixes features and fixes, so stability-minded users must either take risky changes or forgo security patches. And libraries pin themselves to a lowest-common-denominator JDK, often JDK 8.
- **Semantic versioning doesn't fix it**: it labels the kind of change but doesn't let you take fixes without new features.
- **Cascades of change**: a library's bug-fix release often bumps *its* dependencies, dragging major changes through your dependency tree.
- **Multiple release trains are expensive** when lots of fixes are backported between them.

The proposed fix has two kinds of release train. The **tip** *≈ in JEP 14, the single release line that gets new features, enhancements and fixes, and can target a recent JDK* gets nearly all development. Each **tail** *≈ in JEP 14, an older release line that gets only critical bug fixes and security patches, and targets an older JDK* exists to "preserve the status quo".[18] In the JEP's words, "the tip release of a library contains new features and bug fixes, while tail releases contain only critical bug fixes."[18] The discipline lies in the rules: backport as little as possible, choose a JDK baseline per train (a recent JDK for the tip, JDK 17 or 21 for a tail), keep tails cheap so you can afford several, and decline backport requests.[18]



The JDK itself has worked this way since 2018: JDK 17.0.x and 21.0.x are fix-only tails, while the tip moves every six months.[18] JEP 14 also points to Spring Boot as a library already close to the model: new features only go into a time-boxed tip roughly every six months, and some releases (2.7, for example) are maintained as longer tails.[18]

Helidon's road to 27

Helidon announced the switch in April 2026 with version 4.4.0. InfoQ quoted the plan: Helidon "will change its familiar semantic versioning to match the OpenJDK six-month release cadence", and "Helidon 4.4.0 today will change to Helidon 27".[19] So the jump from 4 to 27 isn't 23 major releases. The version number now simply *is* the JDK version.

Helidon, from J4C to the JDK's cadence



Sources: InfoQ (2 Apr and 16 Oct 2026), devops.com (Mar 2026), Helidon 27.0.0 and 4.5.5 changelogs. The Helidon 28 timing is InfoQ's; the next-LTS statement is the changelog's.

HOW IT WORKS

Two lines, two JDK baselines

The practical result is two Helidon lines with different contracts:

- **Helidon 27 (the tip)** needs Java 27 and Maven 3.8 or later.[20] Per the changelog, the minimum Java version will bump with each release. InfoQ expects Helidon 28 alongside JDK 28 in March 2027.[13,12]
- **Helidon 4.5.x (the tail)** keeps Helidon's familiar version numbers, requires Java 21 and recommends Java 25.[16] InfoQ describes it as focused on critical bugs, security fixes and selected backports, for teams that need a long-lived production base.[12]

Even the tail is pointing users forward. Helidon 4.5.5 adds an opt-in `case-sensitive-methods: true` listener option and recommends enabling it "to align applications with future Helidon versions".[16] That's a tail doing its job: no new features, but help with crossing to the tip.

One repository becomes three

The old Helidon repository contained everything. Now it's split into three projects:[13,14]

What used to be one Helidon release

Helidon Core (27.0.0)	imperative + declarative, needs Java 27
Helidon MicroProfile	own repo, 'MP 5', not released yet
Helidon Extensions	per-extension versions (Kafka, OCI …)

Helidon Core is InfoQ's name for what was Helidon SE. Repository state checked 17 Oct 2026: helidon-microprofile has no release tags yet; helidon-extensions has 27.0.0 tags for 13 extensions, including the Kafka, JMS and Pulsar messaging connectors.

Helidon Core replaces Helidon SE and offers two programming models: imperative ("plain Java", no injection or annotations) and declarative.[12] **Helidon MicroProfile** now has its own repository, whose README says "Java 27 is required to use Helidon MicroProfile 5". So it's back on semantic versioning, starting at 5. When we checked on 17 October, it had no release tags yet.[21] **Helidon Extensions** holds the former integrations, each released on its own schedule. It already has 27.0.0 tags for 13 extensions, including Kafka, JMS and Pulsar messaging connectors, Neo4j, HashiCorp Vault and OCI.[22]

The same idea, two Helidon programming models

Imperative (wire it yourself)

```
WebServer.builder()
    .routing(r -> r.get("/greet",
        (req, res) -> res.send("Hello!")))
    .build()
    .start();
```

Declarative (generated at build)

```
@RestServer.Endpoint
@Http.Path("/greet")
@Service.Singleton
class GreetEndpoint {
    @Http.GET
    String greet() { return "Hello!"; }
}
```

Simplified from the Helidon 27 WebServer and Declarative docs. Declarative needs the helidon-bundles-apt annotation processor.

Declarative, without runtime magic

Helidon Declarative ≈ *Helidon's annotation-based programming model. You write annotated classes, and an annotation processor generates the plain Helidon code and wiring at build time* gives you annotation-driven code without runtime reflection. Its documentation explains how: it generates all the necessary code at build time "to avoid reflection and bytecode manipulation at runtime", and the generated code uses Helidon's imperative APIs, so performance matches a hand-written imperative app.[23] It's now a **preview feature** ≈ *in Helidon's terms, production-ready and backward compatible within a major version, but allowed to change in a later major version without the usual deprecation cycle*, and it covers HTTP endpoints, typed HTTP and gRPC clients, fault tolerance, scheduling, validation, security, metrics, tracing, WebSocket, GraphQL, CORS and health checks.[23]

The declarative app starts from a generated class rather than by scanning the classpath: `ServiceRegistryManager.start(ApplicationBinding.create())`, with a `@Service.GenerateBinding` annotation on `main` to "bypass discovery and runtime binding".[23]

Messaging without reactive streams

Helidon Messaging is new, and its design says a lot about where Java has gone since virtual threads. InfoQ notes that it uses ordinary blocking Java and avoids reactive APIs such as MicroProfile Reactive Messaging: you annotate a method, and it's called once per message.[12,24]

```
@Service.Singleton
final class OrderHandler {
    @Messaging.ReceiveFrom("orders")
    void receive(Order order) {
        // Process one order.
    }
}
```

Channels such as `orders` are logical names, bound through configuration to a connector (Kafka, JMS or Pulsar, from Helidon Extensions). The delivery contract is explicit: the default is "synchronous and at least once", so handlers "must tolerate duplicate delivery".[24] A handler can take the payload, a `Message<T>` envelope with headers, or a whole `MessageBatch<T>`.[24] **At-least-once delivery** ≈ *a messaging guarantee that every message is processed, but some may be processed more than once (for example after a crash), so handlers must be idempotent* means your handlers must be idempotent, and it's good to see the guarantee written down so plainly.

Data: two different things with similar names

InfoQ describes "Helidon Data JDBC" as a repository API over ORM.[12] Helidon's own docs show there are really two separate pieces, and the difference matters:[25]

- The **Helidon Data Repository** (preview) is the ORM one. You write an interface annotated with `@Data.Repository`, extend `Data.CrudRepository<Pet, Integer>`, and declare methods like `Optional<Pet> findByName(String name)`, or add a `@Data.Query` with JPQL. The implementation is generated at compile time on top of **Jakarta Persistence** ≈ *the standard Java ORM API (formerly JPA). Hibernate and EclipseLink are implementations*, with EclipseLink or Hibernate.
- **Helidon Data JDBC** (incubating, "not production ready") is the new one in 27. It has no entity management and doesn't derive queries from method names. You write the SQL in a `@Jdbc.Statement` annotation, map results onto a record, and Helidon generates the JDBC code at compile time. There's also an imperative `JdbcClient` for queries built at runtime.

```
@SuppressWarnings(Api.SUPPRESS_INCUBATING)
@Data.Repository
@Data.Provider("jdbc")
@Jdbc.Client("contacts")
public interface ContactRepository {
    @Jdbc.Statement("""
        SELECT ID AS id, NAME AS name
        FROM CONTACT WHERE ID = :id""")
    Optional<Contact> findById(long id);
}
record Contact(long id, String name) {}
```

That `@SuppressWarnings` line points to another change in Helidon 27: **API stability is now enforced by the compiler**. Every public Helidon API is annotated as `@Api.Stable`, `@Api.Preview`, `@Api.Incubating` or `@Api.Internal`,[26] and an annotation processor rejects uses of incubating APIs by default. You opt in per package, type or method with `Api.SUPPRESS_INCUBATING`, or project-wide with `-Ahelidon.api.incubating=warn`.[25] It's the framework equivalent of the JDK's `--enable-preview`: you can't use something unfinished by accident.

WHAT IT MEANS FOR YOU

The upgrade guide is unusually direct, so let's follow it.

If you run Helidon MP (CDI, JAX-RS, MicroProfile), stay on 4.5.x for now. The guide says not to upgrade those workloads to Helidon 27 "unless you are also replacing the MP programming model with Helidon Core APIs", and to wait for the independent MicroProfile release.[14] Since Helidon 4 is the LTS line,[13] staying there isn't a cop-out. It's the supported path.

If you run Helidon SE and you're happy on Java 21 or 25, you can also stay put. That's the tail's job. Just take patch releases such as 4.5.5, and start the preparation the guide recommends on 4.x: fix deprecation warnings and turn on `case-sensitive-methods: true`.[16,14]

If you want the tip, budget for a JDK upgrade every six months. Helidon 27 needs Java 27, and the rule is that the minimum rises with each release.[13] Your toolchains, CI images, container images and native-image builds all have to move with it, and the guide lists every one of them.[14] Remember that Java 27 isn't an LTS release: Oracle's roadmap, for example, shows JavaFX 27 support ending in March 2027, when JDK 28 arrives.[27] Running the tip means always running a recent JDK.

Expect a checklist of breaking changes when you move:[14]

- Remove dependencies on modules that no longer exist (`helidon-microprofile*`, `helidon-integrations*`, the Jaeger, Zipkin and OpenTracing providers, `helidon-metrics-prometheus`, `helidon-cors` and more), and use the replacements (for example `helidon-webserver-cors` and the OpenTelemetry tracing provider).

- Former integrations move from `io.helidon.integrations` to `io.helidon.extensions` coordinates, with a few exceptions (OCI becomes `oci.v3`, Vault becomes `hashicorp.vault`).
- `Config.global(config)` is gone. Use `Services.set(Config.class, config)`.
- `@Service.Provider` is removed. Use an explicit scope such as `@Service.Singleton`.
- Uppercase all HTTP method names in configuration (`GET`, not `get`).

> 🌍 **In the real world: who else works this way?**
>
> JEP 14 itself names two precedents. The JDK has used tip and tail since JDK 10 in 2018, with 17.0.x and 21.0.x as fix-only tails. Spring Boot puts new features only into a six-monthly tip and keeps some releases, like 2.7, alive as longer tails.[18] Helidon takes the model to its strictest form: a tip whose *minimum JDK* is the newest JDK itself. Whether users accept a six-month JDK treadmill in exchange for the newest features is the experiment Helidon is now running for everyone.

> 💥 **Watch out for**
>
> - **"27" isn't a sign of 23 missed releases.** It's the JDK version. Helidon 4.4.0 effectively *became* 27.[19]
> - **No MicroProfile on 27 yet.** The new Helidon MicroProfile repository had no release tags when we checked on 17 October.[21] InfoQ's write-up says MicroProfile "continues to support" the specs, but the changelog and upgrade guide say it isn't part of 27 "at this time".[12,14]
> - **Stability labels are real now.** Data JDBC is incubating: "not production ready", and liable to change or be removed in any release.[25] Messaging and Declarative are preview.[24,23] The compiler will reject incubating APIs until you opt in.[25]
> - **Messaging is at-least-once by default.** Make handlers idempotent. The `DROP` failure policy is an explicit opt-in to discarding messages.[24]
> - **Extensions arrive on their own schedule.** The guide warns that not all extensions "might be available initially". Gson media support, for example, ships separately: the guide planned it for after Helidon 27, and a `gson/27.0.0` extension tag now exists.[14,22]

WHAT HAPPENS NEXT

Three dates are worth watching. First, the Helidon MicroProfile 5 release, which unblocks MP users who want the tip.[21] Second, **Helidon 28 with JDK 28 in March 2027**, the first time the tip has to move to a new JDK on schedule.[12] Third, **Helidon 29**, announced as the next LTS.[13] JDK 29 is planned as the next Java LTS in September 2027.[27] If Helidon 29 lines up with it, Helidon users get a new tail on a new LTS JDK, which is the JEP 14 model working exactly as intended.

📖 Words from this story

Helidon: Oracle's open-source Java microservices framework, launched in 2018.

MicroProfile: Eclipse Foundation specifications for enterprise Java microservices, built on Jakarta EE APIs like CDI and JAX-RS.

Virtual threads: lightweight JVM-managed threads (final in JDK 21) that let blocking code scale.

Tip: in JEP 14, the one release line that gets new features and can target a recent JDK.

Tail: in JEP 14, an older line that gets only critical fixes and security patches, on an older JDK.

Helidon Declarative: Helidon's annotation model, turned into plain Helidon code at build time.

Preview feature (Helidon): production-ready, compatible within a major version, but may change in the next major version without deprecation.

At-least-once delivery: every message gets processed, but possibly more than once, so handlers must be idempotent.

Jakarta Persistence: the standard Java ORM API (formerly JPA). Hibernate and EclipseLink implement it.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **JEP 545, "Faster Startup and Warmup with ZGC", is now targeted to JDK 28 (16 October):** one of the two ZGC JEPs from our issue #8. The companion adaptive-heap-sizing JEP 546 hadn't been announced as targeted on Inside.java by 17 October.[28]
- **JDK 28 adds APIs to see which TLS key exchange was negotiated:** `ExtendedSSLSession.getNamedGroup()` and `getSupportedNamedGroups()` on `SSLSocket`, `SSLServerSocket` and `SSLEngine` (`@since 28`, 12 October), handy for confirming the post-quantum hybrid from our issue #12.[29]
- **Paul Sandoz on Java and AI (15 October):** an Inside.java video on how Amber, Babylon, Valhalla and Panama could make Java a better fit for AI work.[30]
- **The `sun.cpu.isalist` system property is removed from JDK 28 (14 October):** it reported the CPU's supported instruction sets on Windows and on Unix systems with `sysinfo(SI_ISALIST)`.[31]
- **Spring's first monthly "Patch Thursday" release train is next Thursday, 22 October:** releases and CVE fixes for the whole Spring portfolio in one drop (our issue #2).[32]
- **A Bootiful Podcast asks "Can Java be more performant than Rust?" (15 October):** Andrew Lombardi brings his numbers.[33]

1. **openjdk/jdk commit e9222ab "8385697: Add a Pooled Confined Arena"**, Per Minborg, 3 Sep 2026. We read the diff and `ConfinedSegmentPool.java`, `ArenaImpl.java` and `Thread.java` at openjdk/jdk master 93376e4 (16 Oct 2026) with git; `java/lang/foreign/Arena.java` has no pooling-related API change.
2. openjdk/jdk build tags checked with git on 17 Oct 2026: **jdk-28+14** (3 Sep) does not contain e9222ab; **jdk-28+15** (10 Sep) and later do.
3. **Making Arena.ofConfined() Even Cheaper in JDK 28**, Per-Åke Minborg, Inside.java, 5 Oct 2026.
4. **openjdk/jdk commit 7f0d1d9 "8392541: Remove the internal BufferStack and CaptureStateUtil"**, Per Minborg, 14 Oct 2026 (diff read with git; `SharedUtils.newBoundedArena` now uses `Arena.ofConfined()`).
5. **JEP 442: Foreign Function & Memory API (Third Preview)**, OpenJDK (JDK 21).
6. **JEP 412: Foreign Function & Memory API (Incubator)**, OpenJDK (JDK 17; history of JEPs 370, 383, 393 and 389).
7. **Does Java 18 finally have a better alternative to JNI?**, foojay.io (JEP 419, second incubator in JDK 18).
8. **InfoQ's JDK 22 release coverage**, InfoQ, 19 Mar 2024.

9. `jdk/internal/foreign/SegmentFactories.java` (`allocateNativeInternal`) at openjdk/jdk master 93376e4, read with git on 17 Oct 2026.
10. Our test on OpenJDK 21.0.12.1 (FFM is a preview API there, so we used `--enable-preview`), 2-vCPU Intel Xeon sandbox, 17 Oct 2026. The `open`/`errno` example printed `errno = 2`. Timing: a simple loop of 2 million iterations, after 5 warm-up rounds, three measured rounds. Not JMH, so the figures are indicative only.
11. **jline/jline3** `terminal-ffm` module at commit 72c9a4c (11 Oct 2026), read with git: 27 `Arena.ofConfined()` uses in `src/main`, e.g. `CLibrary.java` (`winsize` for `ioctl`, `pollfd` plus captured state for `poll`).
12. **Helidon 27 Introduces Restructured Programming Models and New Messaging and Data Modules**, Michael Redlich, InfoQ, 16 Oct 2026.
13. **Helidon 27.0.0 CHANGELOG**, helidon-io/helidon (tag 27.0.0, commit 143e870, 17 Sep 2026).
14. **Helidon 27 upgrade guide**, helidon-io/helidon at tag 27.0.0.
15. **Helidon 27 Released**, the Helidon team, Inside.java, 22 Sep 2026.
16. **Helidon 4.5.5 CHANGELOG**, helidon-io/helidon (tag 4.5.5, 15 Sep 2026).
17. **Java 26 arrives with AI integration and a new ecosystem portfolio: what it means for DevOps teams**, DevOps.com (coverage of the 17 Mar 2026 Java 26 release).
18. **JEP 14: The Tip & Tail Model of Library Development**, Alex Buckley, Brian Goetz, Ron Pressler, OpenJDK (created 30 Sep 2024).
19. **Helidon 4.4.0 Introduces Alignment with OpenJDK Cadence and Support via Java Verified Portfolio**, Michael Redlich, InfoQ, 2 Apr 2026.
20. **Helidon 27 "Get Started"**, helidon-io/helidon at tag 27.0.0 (Java 27+, Maven 3.8+).
21. **helidon-io/helidon-microprofile** README ("Java 27 is required to use Helidon MicroProfile 5"), checked with git on 17 Oct 2026: no release tags; latest commit af6ceb1, 14 Oct 2026.
22. **helidon-io/helidon-extensions** tags, checked with git on 17 Oct 2026: 27.0.0 tags for crac, eureka, gson, hashicorp-vault, langchain4j, messaging/jms, messaging/jms-javax, messaging/kafka, messaging/pulsar, neo4j, oci-v3, openapi-ui and toml.
23. **Helidon Declarative documentation**, helidon-io/helidon at tag 27.0.0.
24. **Helidon Messaging README**, helidon-io/helidon at tag 27.0.0.
25. Helidon 27 docs at tag 27.0.0: **Data Repository**, **Data JDBC** and **Data JDBC declarative model**.
26. `io.helidon.common.Api`, helidon-io/helidon at tag 27.0.0.
27. **Oracle Java Verified Portfolio Support Roadmap**, Oracle, updated 15 Sep 2026 (JavaFX table: 27 supported until March 2027; 29 (LTS) planned September 2027).
28. **Inside.java front page**, checked 17 Oct 2026 ("JEP targeted to JDK 28: 545: Faster Startup and Warmup with ZGC", 16 Oct 2026).
29. **openjdk/jdk commit 7996ac7 "8388519: Add APIs to get supported and negotiated named groups"**, Artur Barashev, 12 Oct 2026 (read with git).
30. **How Does Java Fit Into the World of AI—and How Can It Become an Even Better Fit?**, Paul Sandoz, Inside.java, 15 Oct 2026.
31. **openjdk/jdk commit c368bac "6340855: Remove sun.cpu.isalist property"**, Daniel Jeliński, 14 Oct 2026 (read with git).
32. **Releasing Spring for Modern Challenges**, Spring blog, 21 Sep 2026.
33. **A Bootiful Podcast: Can Java be more performant than Rust? Andrew Lombardi has the numbers**, Spring blog, 15 Oct 2026.